

MEMORIA SPRINT 2

Grupo 12.08 “LongNight”

Integrantes:

Jorge Carvajal Lara

Sergi Torres Albert

Brad Owen Valladares Quispe

Javier Perepérez Cardona

Índice:

1. Patrón de Diseño Aplicado.....	3
1.1. Identificación y Propósito.....	3
1.2. Estructura de Clases	4
1.3. Implementación	6
1.4. Justificación de Diseño	13
2. Lista de Patrones a Aplicar - Sprints Futuros	15
2.1. Propuesta de Patrón Adapter	15
2.2. Propuesta de Patrón Observer.....	15
2.3. Propuesta de Patrón State	16
2.4. Análisis de Factibilidad	17
3. Refactoring y Tests Unitarios	18
3.1. Listado de Refactorizaciones.....	18
3.2. Pruebas Unitarias	25
4. Decisiones de Diseño tomadas - ADR	27
ADR-005: Estandarización de Rutas e Identificación de Contexto mediante Parámetros de URL	27
ADR-006: Centralización del Estado del Evento mediante Contexto Global	29
ADR-007: Flujo de Onboarding de Jurados basado en Tokens de Invitación	31
ADR-008: Implementación de la extensión Fingerprinting para Unicidad de Voto y Seguridad.....	33
Anexo: Metodología de Generación de los ADR.....	35
5. Uso y Documentación de IA Generativa - AI Usage Log	36
5.1. Herramientas usadas.....	36
5.2. Prompts clave	36
5.3. Salidas relevantes.....	37
5.4. Qué aceptamos y qué rechazamos	38
5.5. Cómo lo verificamos.....	38
5.6. Resultado final / decisión humana.....	39

1. Patrón de Diseño Aplicado

1.1. Identificación y Propósito

Nombre del Patrón: DECORATOR

- **Problema resuelto:** Necesitábamos añadir lógica de validación (comentarios obligatorios, comprobación de configuración del evento) al servicio de votación (VotoService) sin modificar su código original. El Decorator permite envolver el servicio base con capas adicionales de responsabilidad, manteniendo el principio Open/Closed: el servicio original está cerrado a modificación pero abierto a extensión.

Nombre del Patrón: STRATEGY

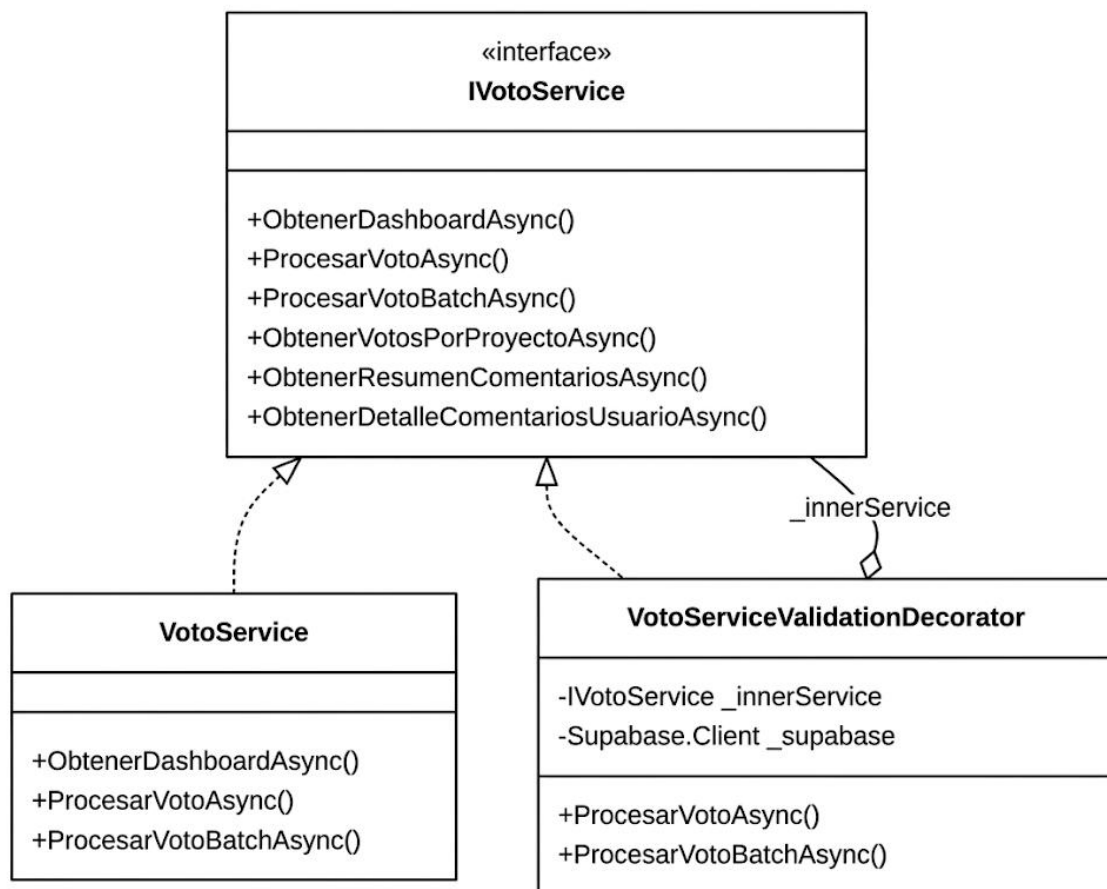
- **Problema resuelto:** La validación de comentarios en los votos varía en tiempo de ejecución según la configuración del evento y de cada criterio de evaluación individual. Algunos eventos/criterios requieren comentarios obligatorios y otros no. El patrón Strategy permite encapsular cada algoritmo de validación en una clase independiente y seleccionar dinámicamente cuál aplicar, evitando cadenas de if/else dispersas por el código.

Nombre del Patrón: FILTER

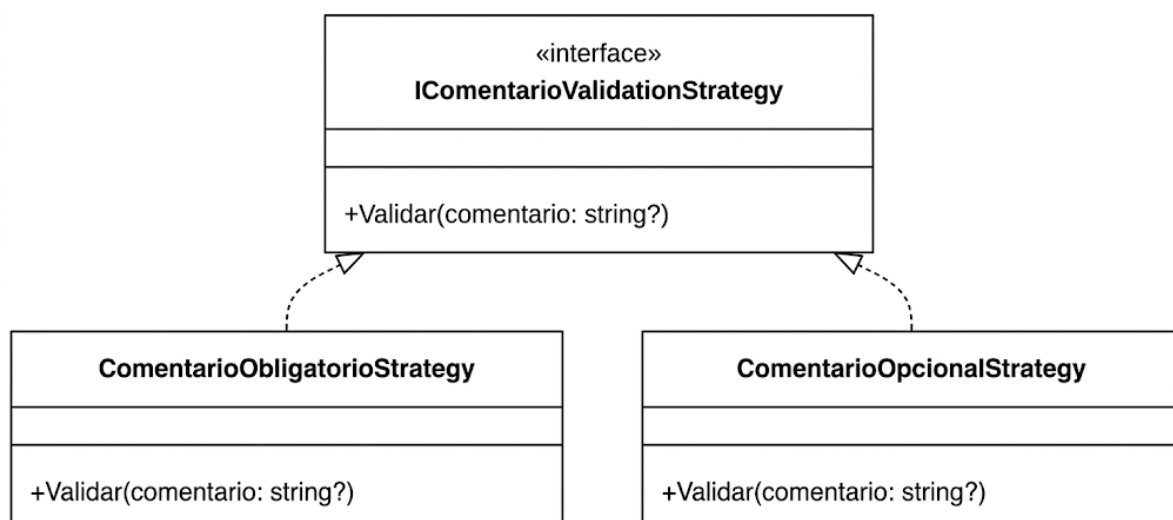
- **Problema resuelto:** Varios endpoints de la API requieren verificar que el usuario autenticado sea Organizador del evento antes de permitir la acción (gestionar jurados, configurar categorías, etc.). Sin un filtro, cada controlador o acción tendría que duplicar la lógica de autorización. El patrón Filter (implementado como IAsyncActionFilter) permite centralizar esta verificación transversal y aplicarla de forma declarativa mediante un atributo OrganizerOnly.

1.2. Estructura de Clases

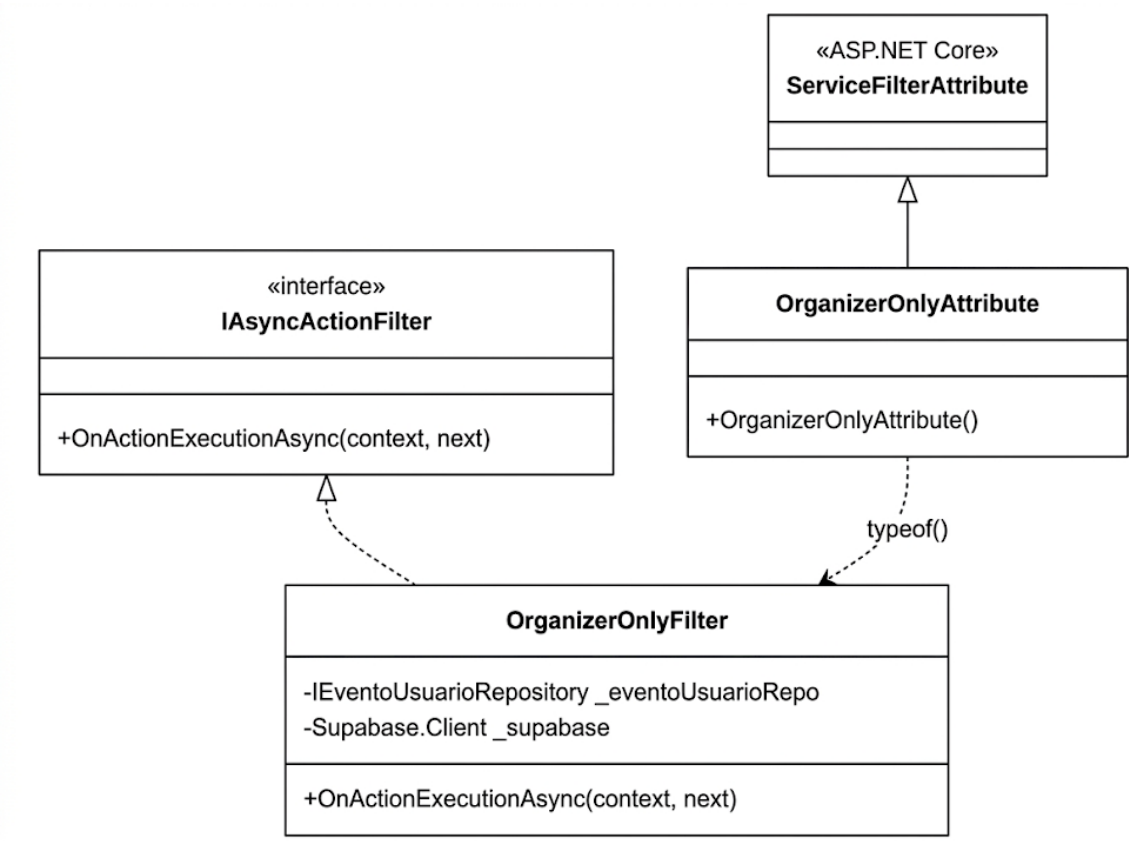
Patrón DECORATOR:



Patrón STRATEGY:



Patrón FILTER:



1.3. Implementación

Patrón DECORATOR:

Archivo: Backend/Votify.API/Services/IVotoService.cs

La interfaz IVotoService es el **contrato común** que tanto el servicio base (VotoService) como el decorador (VotoServiceValidationDecorator) implementan. Esta interfaz es la pieza clave del patrón Decorator, ya que garantiza que ambos componentes sean intercambiables: cualquier consumidor (por ejemplo, un controlador) trabaja contra la interfaz sin conocer si está invocando al servicio real o a su versión decorada. Define los métodos principales del módulo de votación: obtener el dashboard, procesar votos individuales y en lote (batch), y consultar votos y comentarios.

```
using Votify.API.Models.DTOS;

namespace Votify.API.Services
{
    public interface IVotoService
    {
        Task<DashboardResponseDto> ObtenerDashboardAsync(int eventoId, int? idUsuario = null, string? sessionId = null, string? identificadorHash = null);
        Task<DashboardResponseDto> ProcesarVotoAsync(VotoRequestDto request, int? idUsuario = null, string? sessionId = null);
        Task<DashboardResponseDto> ProcesarVotoBatchAsync(VotoBatchRequestDto request);
        Task<IEnumerable<VotoResponseDto>> ObtenerVotosPorProyectoAsync(int proyectoId);
        Task<List<TipoComentaristaDto>> ObtenerResumenComentariosAsync(int proyectoId, int categoriaId);
        Task<List<ComentarioDetalleDto>> ObtenerDetalleComentariosUsuarioAsync(int proyectoId, int categoriaId, string usuarioRef);
    }
}
```

Archivo: Backend/Votify.API/Services/Decorators/VotoServiceValidationDecorator.cs

Esta clase es el **núcleo del patrón Decorator**. Implementa la misma interfaz IVotoService que el servicio original, pero recibe en su constructor una instancia de IVotoService (el servicio «envuelto», almacenado en `_innerService`). Su responsabilidad consiste en **interceptar** las llamadas de votación para ejecutar validaciones previas — en concreto, comprobar si el evento o los criterios exigen comentarios obligatorios — y, solo si las validaciones pasan correctamente, **delegar** la operación al servicio original mediante `_innerService`. Los métodos que no requieren validación adicional (como `ObtenerVotosPorProyectoAsync`) simplemente redirigen la llamada al servicio interno sin añadir lógica, actuando como pass-through transparente.

Los puntos clave a resaltar son:

- **Línea 10:** campo `_innerService` que contiene la referencia al servicio original envuelto.
- **Líneas 48-52:** uso del patrón Strategy dentro del Decorator para seleccionar la validación.
- **Líneas 54-55 y 95:** delegación al servicio original con `_innerService.ProcesarVotoAsync(...)` y `_innerService.ProcesarVotoBatchAsync(...)`.
- **Líneas 98-111:** métodos que simplemente delegan sin añadir lógica (pass-through).

```
public class VotoServiceValidationDecorator : IVotoService
{
    private readonly IVotoService _innerService;
    private readonly Supabase.Client _supabase;

    public VotoServiceValidationDecorator(IVotoService innerService, Supabase.Client supabase)
    {
        _innerService = innerService;
        _supabase = supabase;
    }

    public async Task<DashboardResponseDto> ProcesarVotoAsync(VotoRequestDto request, int? idUsuario =
null, string? sessionId = null)
    {
        // 1. Obtener el evento para saber la configuración
        var eventoResponse = await _supabase.From<EventoLite>()
            .Filter("id", Supabase.Postgrest.Constants.Operator.Equals, request.EventoId.ToString())
            .Get();
        var evento = eventoResponse.Models.FirstOrDefault()
            ?? throw new Exception("Evento no encontrado");

        // 2. Aplicar Patrón Strategy para la validación
        IComentarioValidationStrategy strategy = evento.ComentariosObligatorios
            ? new ComentarioObligatorioStrategy()
            : new ComentarioOpcionalStrategy();

        strategy.Validar(request.Comentario);

        // 3. Si no hay excepción, delegamos al servicio original (el inner)
        return await _innerService.ProcesarVotoAsync(request, idUsuario, sessionId);
    }
    // ... resto de métodos delegados
}
```

Archivo: Backend/Votify.API/Program.cs

Este fragmento muestra cómo se **conecta el Decorator** en el contenedor de inyección de dependencias de ASP.NET Core. Primero se registra VotoService como clase concreta; después, al registrar la interfaz IVotoService, se usa una factory lambda que resuelve el servicio base y lo envuelve con VotoServiceValidationDecorator. De este modo, todo el resto de la aplicación que inyecte IVotoService recibirá automáticamente la versión decorada sin necesidad de cambiar ni una línea en los controladores. Esta configuración es lo que hace transparente el patrón para los consumidores del servicio.

```
// Registro con Decorator para VotoService
builder.Services.AddScoped<Votify.API.Services.VotoService>();
builder.Services.AddScoped<Votify.API.Services.IVotoService>(provider =>
{
    var baseService = provider.GetRequiredService<Votify.API.Services.VotoService>();
    var supabase = provider.GetRequiredService<Supabase.Client>();
    return new Votify.API.Services.Decorators.VotoServiceValidationDecorator(baseService, supabase);
});
```

Patrón STRATEGY:

Archivo: Backend/Votify.API/Models/Domain/Strategies/IComentarioValidationStrategy.cs

A continuación, se muestran los tres archivos que componen la familia de estrategias. La interfaz `IComentarioValidationStrategy` define un único método `Validar()` que recibe el texto del comentario. Cada estrategia concreta encapsula un comportamiento de validación diferente: `ComentarioObligatorioStrategy` lanza una excepción si el comentario está vacío (impidiendo que el voto se procese), mientras que `ComentarioOpcionalStrategy` permite que el comentario sea nulo o vacío sin ningún impedimento. Esta separación permite que el sistema elija en tiempo de ejecución qué regla aplicar según la configuración del evento o criterio, sin recurrir a condicionales `if/else` dentro de la lógica de votación.

```
namespace Votify.API.Models.Domain.Strategies
{
    public interface IComentarioValidationStrategy
    {
        void Validar(string? comentario);
    }
}
```

Archivo: Backend/Votify.API/Models/Domain/Strategies/ComentarioObligatorioStrategy.cs

```
namespace Votify.API.Models.Domain.Strategies
{
    public class ComentarioObligatorioStrategy : IComentarioValidationStrategy
    {
        public void Validar(string? comentario)
        {
            if (string.IsNullOrEmpty(comentario))
            {
                throw new ArgumentException("El comentario es obligatorio para evaluar en este evento.");
            }
        }
    }
}
```


Archivo: Backend/Votify.API/Models/Domain/Strategies/ComentarioOpcionalStrategy.cs

```
namespace Votify.API.Models.Domain.Strategies
{
    public class ComentarioOpcionalStrategy : IComentarioValidationStrategy
    {
        public void Validar(string? comentario)
        {
            // El comentario es opcional, no hacemos validación estricta
        }
    }
}
```

Archivo: Backend/Votify.API/Services/Decorators/VotoServiceValidationDecorator.cs

Este fragmento es especialmente relevante porque muestra la **sinergia entre los patrones Decorator y Strategy en acción**. Dentro del método `ProcesarVotoBatchAsync` del Decorator, se itera sobre cada evaluación (criterio) del voto batch y se consulta en la base de datos si ese criterio tiene el flag `ComentarioObligatorio` activado. En función de ese valor, se instancia dinámicamente la estrategia correspondiente (`ComentarioObligatorioStrategy` o `ComentarioOpcionalStrategy`) y se ejecuta su método `Validar()`. Además, se aplica una segunda validación a nivel global del evento para el comentario general del proyecto. Este diseño demuestra la granularidad del sistema: cada criterio puede tener su propia política de validación independiente del resto.

```
public async Task<DashboardResponseDto> ProcesarVotoBatchAsync(VotoBatchRequestDto request)
{
    // ...obtener evento y criterios...

    // 3. Validar comentarios POR CRITERIO: solo los que tienen comentario_obligatorio = true
    foreach (var evaluacion in request.Evaluaciones)
    {
        var criterioBd = criteriosDb.FirstOrDefault(c => c.Id == evaluacion.CriterioId);

        // Solo el flag individual del criterio determina si SU comentario es obligatorio
        bool requiereComentario = criterioBd?.ComentarioObligatorio ?? false;

        IComentarioValidationStrategy strategy = requiereComentario
            ? new ComentarioObligatorioStrategy()
            : new ComentarioOpcionalStrategy();

        strategy.Validar(evaluacion.Comentario);
    }

    // 4. Validar COMENTARIO GLOBAL: solo si el evento lo exige a nivel global
    if (evento.ComentariosObligatorios)
    {
        IComentarioValidationStrategy globalStrategy = new ComentarioObligatorioStrategy();
        globalStrategy.Validar(request.ComentarioGlobal);
    }

    // 5. Delegar al servicio original
    return await _innerService.ProcesarVotoBatchAsync(request);
}
```

Patrón FILTER:

Archivo: Backend/Votify.API/Filters/OrganizerOnlyAttribute.cs

Esta clase es un wrapper ligero que hereda de `ServiceFilterAttribute` de ASP.NET Core. Su único propósito es facilitar la aplicación del filtro con una sintaxis limpia y legible: `OrganizerOnly` en lugar de `ServiceFilter(typeof(OrganizerOnlyFilter))`. Al heredar de `ServiceFilterAttribute`, permite que el filtro real (`OrganizerOnlyFilter`) sea resuelto por el contenedor de inyección de dependencias, habilitando la inyección de repositorios y servicios necesarios para la verificación de permisos.

```
using Microsoft.AspNetCore.Mvc;

namespace Votify.API.Filters
{
    public class OrganizerOnlyAttribute : ServiceFilterAttribute
    {
        public OrganizerOnlyAttribute() : base(typeof(OrganizerOnlyFilter))
        {
        }
    }
}
```

Archivo: Backend/Votify.API/Filters/OrganizerOnlyFilter.cs

Esta clase contiene toda la lógica de autorización centralizada. Implementa `IAsyncActionFilter`, lo que le permite interceptar cualquier petición HTTP antes de que llegue al controlador. El flujo de verificación sigue una cadena de comprobaciones secuenciales: primero extrae y valida el token JWT de la cabecera `Authorization`; después obtiene la identidad del usuario desde `Supabase Auth`; a continuación, extrae el `IdEvento` de forma inteligente (buscando tanto en los parámetros de ruta como en el body de la petición mediante `reflection`); y finalmente consulta la relación `EventoUsuario` en la base de datos para confirmar que el usuario tiene el rol de «Organizador». Si cualquier paso falla, el filtro cortocircuita la petición devolviendo un 401 (no autenticado) o 403 (sin permisos) sin que el controlador llegue a ejecutarse. Solo si todas las verificaciones pasan, se invoca `await next()` para continuar con la ejecución normal.

```
public class OrganizerOnlyFilter : IAsyncActionFilter
{
    private readonly IEventoUsuarioRepository _eventoUsuarioRepo;
    private readonly Supabase.Client _supabase;

    public OrganizerOnlyFilter(IEventoUsuarioRepository eventoUsuarioRepo, Supabase.Client supabase)
    {
        _eventoUsuarioRepo = eventoUsuarioRepo;
        _supabase = supabase;
    }

    public async Task OnActionExecutionAsync(ActionExecutingContext context, ActionExecutionDelegate
next)
    {
        // 1. Obtener el token de la cabecera Authorization
        string? token =
context.HttpContext.Request.Headers["Authorization"].ToString()?.Replace("Bearer ", "");
        if (string.IsNullOrEmpty(token))
        {
            context.Result = new UnauthorizedResult();
            return;
        }

        // 2. Obtener el usuario autenticado desde Supabase Auth
        var user = await _supabase.Auth.GetUser(token);
        if (user == null || string.IsNullOrEmpty(user.Email))
        {
            context.Result = new UnauthorizedResult();
            return;
        }

        // 3. Extraer idEvento de la ruta o del body
        // ... (extracción dinámica desde RouteData o ActionArguments)

        // 4. Verificar si el usuario es Organizador del evento
        var relacion = await _eventoUsuarioRepo.GetAsync(idEvento.Value, dbUserResponse.Id);
        if (relacion == null || relacion.Rol != "Organizador")
        {
            context.Result = new ObjectResult(new { message = "No tienes permisos de organizador..." })
{ StatusCode = 403 };
            return;
        }

        await next(); // Continúa la ejecución del controlador
    }
}
```

Archivo: Backend/Votify.API/Controllers/JuradoController.cs

Esta captura muestra el resultado práctico del patrón Filter: la aplicación del atributo `OrganizerOnly`] a nivel de clase en el controlador `JuradoController`. Al colocarlo junto a `ApiController` y `Route`, todos los endpoints del controlador (asignar jurado, reenviar invitación, listar jurados, eliminar jurado) quedan automáticamente protegidos por la verificación de organizador, sin necesidad de añadir ninguna lógica de autorización dentro de los métodos de acción. Esto demuestra la potencia del patrón: una sola línea de atributo reemplaza decenas de líneas de código de autorización que de otro modo habría que duplicar en cada endpoint.

```
using Microsoft.AspNetCore.Mvc;
using Votify.API.Services;
using Votify.API.Models.Domain;
using Votify.API.Models.DTOS;
using Votify.API.Filters;

namespace Votify.API.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    [OrganizerOnly] // ← Se aplica a TODOS los endpoints del controlador
    public class JuradoController : ControllerBase
    {
        private readonly IJuradoService _juradoService;

        public JuradoController(IJuradoService juradoService)
        {
            _juradoService = juradoService;
        }
        // ...
    }
}
```

1.4. Justificación de Diseño

Patrón DECORATOR:

Se decidió usar el patrón Decorator en lugar de heredar de VotoService o modificarlo directamente por las siguientes razones:

1. **Principio Open/Closed:** VotoService contiene la lógica de negocio de votación (creación de votos, gestión de dashboard, etc.) y no debe ser modificado cada vez que se añadan nuevas validaciones. El Decorator permite añadir validaciones (como la obligatoriedad de comentarios) sin tocar ni una línea de VotoService.
2. **Composición sobre herencia:** El decorador envuelve la interfaz IVotoService, no la clase concreta. Esto permite apilar múltiples decoradores en el futuro (p.ej., un decorador de logging, un decorador de auditoría) sin crear jerarquías de herencia complejas.
3. **Transparencia:** El resto de la aplicación (controllers) consume IVotoService sin saber si detrás hay un servicio directo o un decorador, gracias a la inyección de dependencias configurada en Program.cs.
4. **Integración con Strategy:** El Decorator aprovecha internamente el patrón Strategy para decidir qué validación aplicar, creando una sinergia entre ambos patrones.

Patrón STRATEGY:

1. **Eliminación de condicionales dispersos:** Sin Strategy, la validación de comentarios obligatorios/opcionales requeriría múltiples bloques if/else dentro de los métodos de votación. Con Strategy, cada regla de validación se encapsula en su propia clase.
2. **Extensibilidad:** Si en el futuro se necesita una nueva política (p.ej., comentario con longitud mínima, comentario con formato específico), basta con crear una nueva clase que implemente IComentarioValidationStrategy sin tocar las existentes.
3. **Granularidad a dos niveles:** El patrón se aplica tanto a nivel de criterio individual (ComentarioObligatorio de cada Criterio) como a nivel de evento global (ComentariosObligatorios del Evento), demostrando la flexibilidad de seleccionar estrategias dinámicamente.
4. **Sinergia con Decorator:** El Strategy se invoca dentro del Decorator, lo que separa responsabilidades: el Decorator se encarga de *cuándo* validar (interceptando la llamada) y el Strategy de *cómo* validar (regla concreta).

Patrón FILTER:

1. **Separación de preocupaciones (Cross-Cutting Concern):** La autorización por rol es una preocupación transversal que no pertenece a la lógica de negocio de cada controlador. El filtro la centraliza en un único punto.
2. **Declarativo con atributo:** Se creó OrganizerOnlyAttribute como wrapper de ServiceFilterAttribute para poder aplicar el filtro con la sintaxis limpia [OrganizerOnly] en vez de [ServiceFilter(typeof(OrganizerOnlyFilter))].
3. **Inyección de dependencias:** Al usar ServiceFilterAttribute, el filtro se resuelve desde el contenedor de DI (registrado en Program.cs línea 52), lo que permite inyectar IEventoUsuarioRepository y Supabase.Client sin recurrir a service locators.
4. **Reutilización:** Cualquier controlador que necesite protección de organizador solo necesita añadir [OrganizerOnly] a nivel de clase o de acción individual, sin duplicar ninguna lógica.
5. **Extracción inteligente del IdEvento:** El filtro intenta extraer el ID del evento tanto de la ruta (RouteData) como del body de la petición (usando reflection), lo que lo hace reutilizable en endpoints con diferentes firmas.

2. Lista de Patrones a Aplicar - Sprints Futuros

2.1. Propuesta de Patrón Adapter

Problema a resolver: Nuestra aplicación necesita integrar diversas funcionalidades de IA, como la síntesis de comentarios, proporcionadas por la API de Gemini. Sin el patrón Adapter, la lógica de negocio quedaría acoplada directamente a la interfaz específica de Gemini, dificultando cambiar o añadir otras APIs de IA en el futuro.

Escenario de uso: Se implementará un GeminiApiAdapter que adaptará la interfaz genérica de nuestro servicio de IA (IComentarioSynthesizer) a la interfaz específica de la API de Gemini. Esto permitirá que el servicio de síntesis invoque Gemini de forma transparente, traduciendo solicitudes y respuestas entre ambos formatos.

Justificación: El patrón Adapter permite que los componentes existentes interactúen con la API de Gemini sin modificar su código. Además, reduce el acoplamiento con dependencias externas, favorece el principio Open/Closed y facilita sustituir la implementación de IA por otra en el futuro sin afectar al resto de la aplicación.

2.2. Propuesta de Patrón Observer

Problema a resolver: Existen varios componentes de la interfaz, como el ranking, los comentarios o el feed de actividad, que dependen de cambios en los datos de votaciones, comentarios y proyectos. Actualmente, estas actualizaciones requieren recargas manuales o lógica explícita de sincronización.

Escenario de uso: Se utilizará para notificar automáticamente a los componentes de la interfaz cuando cambien datos relevantes, como nuevas votaciones o comentarios.

Justificación: Cuando se registre una nueva votación en el backend, componentes como el ranking o el feed de actividad podrán actualizarse automáticamente sin necesidad de recargar la página.

2.3. Propuesta de Patrón State

Problema a resolver: La gestión de estados en entidades con ciclos de vida complejos, como Eventos o Proyectos, suele generar gran cantidad de condicionales (`if/else` o `switch`) dispersos por la lógica de negocio. Esto dificulta mantener el código y añadir nuevos estados o comportamientos.

Escenario de uso:

- Dashboard del Organizador: Gestión de Proyectos: Cada estado del proyecto (En Revisión, Aceptado, Rechazado, etc.) definiría las acciones permitidas y su comportamiento mediante objetos de estado específicos.
- Inscripción y Gestión de Eventos para Participantes: El objeto Evento delegaría comportamientos como permitir inscripciones o iniciar votaciones a distintos estados concretos según la fase actual del evento.

Justificación:

Mejora la claridad y mantenibilidad al centralizar la lógica de cada estado en clases independientes.

Facilita añadir nuevos estados o modificar comportamientos sin alterar la lógica principal, siguiendo el principio Open/Closed.

Reduce errores lógicos relacionados con transiciones o acciones inválidas en determinados estados.

2.4. Análisis de Factibilidad

Los patrones Adapter, Observer y State son viables de implementar en los próximos sprints y encajan de forma natural con la arquitectura actual del proyecto, aportando soluciones modulares y escalables.

- **El patrón Adapter** puede integrarse principalmente en el backend como intermediario entre la aplicación y la API de Gemini, permitiendo una integración desacoplada y flexible con servicios externos de IA.
- **El patrón Observer** puede implementarse en el frontend aprovechando mecanismos ya presentes en React, como el manejo de estado y efectos, o mediante una pequeña abstracción para gestionar suscripciones a cambios de datos. Esto permitirá actualizar automáticamente la interfaz ante cambios relevantes.
- **El patrón State** sería útil principalmente para gestionar entidades con ciclos de vida complejos, como Eventos y Proyectos. Su implementación permitiría centralizar el comportamiento asociado a cada estado, eliminar condicionales complejos y facilitar la extensión de nuevos estados o transiciones.

Dado que estos patrones mejoran funcionalidades ya existentes como integración con IA, rankings, votaciones, comentarios y gestión de estados, su implementación resulta realista dentro del tiempo disponible para el próximo sprint.

3. Refactoring y Tests Unitarios

3.1. Listado de Refactorizaciones

Refactorización 1: Extract class

Antes: En EventService teníamos en el constructor el el Supabase.Client y en varios métodos teníamos queries para acceder a base de datos. Un ejemplo:

```
0 references
public EventService(Supabase.Client supabase, ICategoriaRepository categoriaRepository, IProyectoRepository proyectoRepository)
{
    _supabase = supabase;
    _categoriaRepository = categoriaRepository;
    _proyectoRepository = proyectoRepository;
}

2 references
public async Task<List<EventoResponseDto>> GetEventosByUsuarioAsync(int userId)
{
    try
    {
        // Buscar en la tabla relación evento_usuario los eventos de este usuario
        var relaciones = await _supabase
            .From<EventoUsuario>()
            .Filter("idusuario", Supabase.Postgrest.Constants.Operator.Equals, userId.ToString())
            .Get();

        var resultado = new List<EventoResponseDto>();

        foreach (var rel in relaciones.Models)
        {
            // ...
        }
    }
}
```

Después: Ahora se usan los repositorios necesarios manteniendo separada la capa de persistencia.

```
1 reference
public EventService(
    ICategoriaRepository categoriaRepository,
    IProyectoRepository proyectoRepository,
    IEventoRepository eventoRepository,
    IEventoUsuarioRepository eventoUsuarioRepository,
    IBaremoRepository baremoRepository)
{
    _categoriaRepository = categoriaRepository;
    _proyectoRepository = proyectoRepository;
    _eventoRepository = eventoRepository;
    _eventoUsuarioRepository = eventoUsuarioRepository;
    _baremoRepository = baremoRepository;
}
```

```

public async Task<List<EventoResponseDto>> GetEventosByUsuarioAsync(int userId)
{
    try
    {
        var relaciones = await _eventoUsuarioRepository.GetByUsuarioAsync(userId);
        var resultado = new List<EventoResponseDto>();

        foreach (var rel in relaciones)
        {
            var evento = await _eventoRepository.GetByIdAsync(rel.IdEvento);
            if (evento != null)
            {
                resultado.Add(new EventoResponseDto
                {
                    Id = evento.Id,
                    CodEvento = evento.CodEvento,
                    Nombre = evento.Nombre,
                    Descripcion = evento.Descripcion,
                });
            }
        }
    }
}

```

Beneficio: Al crear los repositorios necesarios, estamos extrayendo la lógica de persistencia a una nueva clase independiente.

Refactorización 2: Move Method

Antes: El VotoService tenía la responsabilidad de saber cómo formatear las claves de sesión (U:id o P:session). Esto es un detalle de implementación de "infraestructura" de sesión que ensuciaba la lógica de negocio.

```
3 references
private static string ObtenerClaveSesion(int? idUsuario, string? sessionId)
{
    if (idUsuario.HasValue)
    {
        return $"U:{idUsuario.Value}";
    }

    var safeSessionId = string.IsNullOrEmpty(sessionId) ? "anon-default" : sessionId.Trim();
    return $"P:{safeSessionId}";
}
```

Después: Se ha creado una nueva clase que se encarga de decidir la clave de sesión, y así puede ser reutilizado.

```
namespace Votify.API.Helpers
{
    0 references
    public static class SessionHelper
    {
        /// <summary>
        /// Genera una clave única para identificar la sesión de votación.
        /// Jurado => "U:{idUsuario}"
        /// Público => "P:{sessionId}"
        /// </summary>
        0 references
        public static string ObtenerClaveSesion(int? idUsuario, string? sessionId)
        {
            if (idUsuario.HasValue)
            {
                return $"U:{idUsuario.Value}";
            }

            var safeSessionId = string.IsNullOrEmpty(sessionId) ? "anon-default" : sessionId.Trim();
            return $"P:{safeSessionId}";
        }
    }
}
```

Beneficio: Ahora, se decide cambiar cómo se identifican las sesiones (por ejemplo, usando un prefijo diferente), solo tendremos que tocar un archivo. El VotoService ahora es más "limpio" y delega esa tarea.

Refactorización 3: Extract Method

Antes: El método `ObtenerDashboardAsync` era demasiado largo y difícil de seguir porque incluía una lógica compleja de if-else para determinar qué votos tenía el usuario según si era Jurado, Público con Hash o Público con PIN.

```
public async Task<DashboardResponseDto> ObtenerDashboardAsync(int eventoId, int? idUsuario = null, string? sessionId = null, string? idC
    var claveSesion = ObtenerClaveSesion(idUsuario, sessionId);

    // 1. Obtener votos realizados por este usuario
    List<(int CategoriaId, int ProyectoId)> votosUsuario;

    if (idUsuario.HasValue)
    {
        // Jurado flow: persistente en BD
        if (!VotosRealizadosPorUsuario.ContainsKey(claveSesion) || idUsuario != _ultimoIdUsuarioActivo)
        {
            var votosDb = await _votoRepository.ObtenerVotosDeUsuarioAsync(idUsuario.Value);
            VotosRealizadosPorUsuario[claveSesion] = votosDb
                .Select(v => (v.IdCategoria, v.IdProyecto))
                .Distinct()
                .ToList();
        }
        _ultimoIdUsuarioActivo = idUsuario;
        votosUsuario = VotosRealizadosPorUsuario[claveSesion];
    }
    else if (!string.IsNullOrEmpty(identificadorHash))
    {
        // Public flow with fingerprint: check persistency in registry table
        // Note: Since public voting is one vote per category, we might need a way to track which projects were voted if we wanted
        // But for now, let's just mark the category state correctly.

        // TODO: Could optimize this by adding a repository method to get all voted categories for a hash
        // For now, we'll keep it simple and just use it to mark the status later in the loop if needed.
        votosUsuario = VotosRealizadosPorUsuario.ContainsKey(claveSesion)
            ? VotosRealizadosPorUsuario[claveSesion]
            : new List<(int CategoriaId, int ProyectoId)>();
    }
    else
    {
        // PIN flow (old): stateless memory
        if (!VotosRealizadosPorUsuario.ContainsKey(claveSesion))
        {
            VotosRealizadosPorUsuario[claveSesion] = new List<(int CategoriaId, int ProyectoId)>();
        }
        _ultimoIdUsuarioActivo = null;
        votosUsuario = VotosRealizadosPorUsuario[claveSesion];
    }

    // Obtener datos del evento (sin cache)
    var (categoriasDelEvento, todosProyectos) = await ObtenerDatosEventoAsync(eventoId);
```

Después: Se creo un nuevo método llamado DeterminarVotosUsuarioAsync.

```
private async Task<List<(int CategoriaId, int ProyectoId)>> DeterminarVotosUsuarioAsync(int? idUsuario, string claveSesion, string? identificadorHash)
{
    if (idUsuario.HasValue)
    {
        if (!VotosRealizadosPorUsuario.ContainsKey(claveSesion) || idUsuario != _ultimoIdUsuarioActivo)
        {
            var votosDb = await _votoRepository.ObtenerVotosDeUsuarioAsync(idUsuario.Value);
            VotosRealizadosPorUsuario[claveSesion] = votosDb
                .Select(v => (v.IdCategoria, v.IdProyecto))
                .Distinct()
                .ToList();
        }
        _ultimoIdUsuarioActivo = idUsuario;
        return VotosRealizadosPorUsuario[claveSesion];
    }

    if (!string.IsNullOrEmpty(identificadorHash))
    {
        return VotosRealizadosPorUsuario.ContainsKey(claveSesion)
            ? VotosRealizadosPorUsuario[claveSesion]
            : new List<(int CategoriaId, int ProyectoId)>();
    }

    if (!VotosRealizadosPorUsuario.ContainsKey(claveSesion))
    {
        VotosRealizadosPorUsuario[claveSesion] = new List<(int CategoriaId, int ProyectoId)>();
    }
    _ultimoIdUsuarioActivo = null;
    return VotosRealizadosPorUsuario[claveSesion];
}
```

Beneficio: El flujo principal de ObtenerDashboardAsync ahora es mucho más lineal y fácil de leer. Se puede entender qué hace el dashboard sin perderse en los detalles de cómo se recuperan los votos de la sesión.

Refactorización 4: Inline Temp

Antes: Creamos un variable temporal innecesaria.

```
public async Task<IEnumerable<VotoResponseDto>> ObtenerVotosPorProyectoAsync(int proyectoId)
{
    var votos = await _votoRepository.ObtenerPorProyectoIdAsync(proyectoId);

    var votosPorProyecto = votos.Select(v => new VotoResponseDto
    {
        Id = v.Id,
        ProyectoId = v.IdProyecto,
        CategoriaId = v.IdCategoria,
        Comentario = v.Comentario,
        Fecha = v.FechaVoto,
    });

    return votosPorProyecto;
}
```

Después: El tratamiento ahora es directo.

```
public async Task<IEnumerable<VotoResponseDto>> ObtenerVotosPorProyectoAsync(int proyectoId)
{
    var votos = await _votoRepository.ObtenerPorProyectoIdAsync(proyectoId);

    return votos.Select(v => new VotoResponseDto
    {
        Id = v.Id,
        ProyectoId = v.IdProyecto,
        CategoriaId = v.IdCategoria,
        Comentario = v.Comentario,
        Fecha = v.FechaVoto,
    });
}
```

Beneficio: Nos ahorramos variables redundantes-

Refactorización 5: Introduce Explaining variable

Antes: Las condicionales son muy complejas.

```
if (estadoReal == "Pendiente" && cat.FechaIni.HasValue && nowTime >= cat.FechaIni.Value)
{
    estadoReal = "Activa";
    // Actualizamos en BD para que sea persistente
    await _categoriaRepository.ActualizarAsync(new Categoria { Id = cat.Id, Nombre = cat.Nombre, IdEvento = cat.IdEvento, F
}
else if ((estadoReal == "Activa" || estadoReal == "Pausada") && cat.FechaFin.HasValue && nowTime >= cat.FechaFin.Value)
{
    estadoReal = "Finalizada";
    // Actualizamos en BD para que sea persistente
    await _categoriaRepository.ActualizarAsync(new Categoria { Id = cat.Id, Nombre = cat.Nombre, IdEvento = cat.IdEvento, F
}
```

Después: Se las reduce a variables más claras.

```
bool haLlegadoLaFechaDeInicio = cat.FechaIni.HasValue && nowTime >= cat.FechaIni.Value;
bool haLlegadoLaFechaDeFin = cat.FechaFin.HasValue && nowTime >= cat.FechaFin.Value;
bool estaEnFaseDeVotacion = estadoReal == "Activa" || estadoReal == "Pausada";

if (estadoReal == "Pendiente" && haLlegadoLaFechaDeInicio)
{
    estadoReal = "Activa";
    // Actualizamos en BD para que sea persistente
    await _categoriaRepository.ActualizarAsync(new Categoria { Id = cat.Id, Nombre = cat.Nombre, IdEvento = cat.I
}
else if (estaEnFaseDeVotacion && haLlegadoLaFechaDeFin)
{
    estadoReal = "Finalizada";
    // Actualizamos en BD para que sea persistente
    await _categoriaRepository.ActualizarAsync(new Categoria { Id = cat.Id, Nombre = cat.Nombre, IdEvento = cat.I
}
```

Beneficio: Ahora las condiciones complejas de tiempo y estados tienen nombres claros que explican su propósito sin necesidad de comentarios adicionales.

3.2. Pruebas Unitarias

Prueba 1:

- **Funcionalidad testeada:** Se ha probado la funcionalidad de todo el eventService.cs después de desacoplar las llamadas a base de datos. Ya se respeta la capa de persistencia.
- **Evidencias:** Sé que no está en blanco pero creo que se ve mejor así en negro y no es código que haya que leer y entender.

```
PS C:\Users\bradv\Desktop\proyectos\Votify\TrabajoPSW-DDS-Grupo12.08> dotnet test Backend\Votify.Tests\Votify.Tests.csproj --filter "FullyQualifiedName~EventoServiceTests"
```

```
>>
```

```
Restauración completada (1,5s)
```

```
Votify.API realizado correctamente (1,2s) → Backend\Votify.API\bin\Debug\net9.0\Votify.API.dll
```

```
Votify.Tests realizado correctamente (0,6s) → Backend\Votify.Tests\bin\Debug\net9.0\Votify.Tests.dll
```

```
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v2.8.2+699d445a1a (64-bit .NET 9.0.10)
```

```
[xUnit.net 00:00:00.22] Discovering: Votify.Tests
```

```
[xUnit.net 00:00:00.31] Discovered: Votify.Tests
```

```
[xUnit.net 00:00:00.32] Starting: Votify.Tests
```

```
[xUnit.net 00:00:00.82] Finished: Votify.Tests
```

```
Votify.Tests pruebarealizado correctamente (3,0s)
```

```
Resumen de pruebas: total: 5; con errores: 0; correcto: 5; omitido: 0; duración: 3,0 s
```

```
Compilación realizado correctamente en 8,4s
```

- **Ubicación en el repositorio:** Backend/Votify.Test /EventoServiceTests

Prueba 2:

- **Funcionalidad testeada:** Verificamos que votoService tras la refactorización del inline Temp, extract method , move method e introduce explaining variable siga funcionando igual que antes. Además, aprovechamos y verificamos que el decorador funciona.

- **Evidencias:**

```
PS C:\Users\bradv\Desktop\proyectos\Votify\TrabajoPSW-DDS-Grupo12.08> dotnet test Backend\Votify.Tests\Votify.Tests.csproj --filter "FullyQualifiedName~VotoServiceRefactorTests"
```

```
>>
```

```
Restauración completada (1,0s)
```

```
Votify.API realizado correctamente (0,8s) → Backend\Votify.API\bin\Debug\net9.0\Votify.API.dll
```

```
Votify.Tests realizado correctamente (0,4s) → Backend\Votify.Tests\bin\Debug\net9.0\Votify.Tests.dll
```

```
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v2.8.2+699d445a1a (64-bit .NET 9.0.10)
```

```
[xUnit.net 00:00:00.15] Discovering: Votify.Tests
```

```
[xUnit.net 00:00:00.21] Discovered: Votify.Tests
```

```
[xUnit.net 00:00:00.22] Starting: Votify.Tests
```

```
[xUnit.net 00:00:00.48] Finished: Votify.Tests
```

```
Votify.Tests pruebarealizado correctamente (2,0s)
```

```
Resumen de pruebas: total: 3; con errores: 0; correcto: 3; omitido: 0; duración: 2,0 s
```

```
Compilación realizado correctamente en 5,7s
```

- **Ubicación en el repositorio:** Backend/Votify.Test /VotoServiceRefactorTests

Prueba 3

- **Funcionalidad testeada:** Comprobamos si funciona el patrón strategy.
- **Evidencias:**

```
Tests\Votify.Tests.csproj --filter "FullyQualifiedName~ComentarioValidationTests"
```

```
>> C:\Users\bradv\Desktop\proyectos\Votify\TrabajoPSW-DDS-Grupo12.08>
```

```
Restauración completada (1,3s)
```

```
Votify.API realizado correctamente (1,1s) → Backend\Votify.API\bin\Debug\net9.0\Votify.API.dll
```

```
Votify.Tests realizado correctamente (0,5s) → Backend\Votify.Tests\bin\Debug\net9.0\Votify.Tests.dll
```

```
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v2.8.2+699d445a1a (64-bit .NET 9.0.10)
```

```
[xUnit.net 00:00:00.17] Discovering: Votify.Tests
```

```
[xUnit.net 00:00:00.24] Discovered: Votify.Tests
```

```
[xUnit.net 00:00:00.26] Starting: Votify.Tests
```

```
[xUnit.net 00:00:00.39] Finished: Votify.Tests
```

```
Votify.Tests pruebarealizado correctamente (2,3s)
```

```
Resumen de pruebas: total: 3; con errores: 0; correcto: 3; omitido: 0; duración: 2,2 s
```

```
Compilación realizado correctamente en 6,9s
```

- **Ubicación en el repositorio:** Backend/Votify.Test /ComentarioValidationTests

4. Decisiones de Diseño tomadas - ADR

ADR-005: Estandarización de Rutas e Identificación de Contexto mediante Parámetros de URL

Fecha: 20-04-2026

Sprint: S2

Estado: Aprobado

1) Contexto

Durante el Sprint 1, la aplicación dependía de localStorage para persistir el identificador del evento activo. Este enfoque presentaba deficiencias arquitectónicas críticas:

- **Volatilidad del estado:** El refresco de página provocaba inconsistencias si el almacenamiento local no se sincronizaba correctamente, derivando en errores de ejecución (*undefined*) en las peticiones a la API.
- **Inexistencia de Deep Linking:** La incapacidad de acceder a recursos específicos mediante la URL impedía compartir enlaces directos a paneles de gestión concretos.
- **Acoplamiento indirecto:** Los componentes dependían de un estado global oculto y no declarado, dificultando la trazabilidad de los datos.

2) Opciones consideradas

- **Opción A (No hacer nada):** Continuar usando localStorage y añadir comprobaciones manuales en cada componente para evitar errores de "undefined".
- **Opción B (Context API puro):** Usar un Contexto de React para el evento. Soluciona una parte pero no soluciona el problema de las recargas de página.
- **Opción C (Rutas Dinámicas con Parámetros):** Adoptar el estándar utilizando parámetros de ruta (ej. /eventos/:eventoid/...) como la fuente de verdad primaria para identificar el recurso.

3) Criterios de decisión

- **Robustez:** Resiliencia ante recargas de página y navegación nativa del navegador.
- **Usabilidad técnica:** Soporte para marcadores y compartición de enlaces.
- **Mantenibilidad:** Alineación con los estándares de navegación en Single Page Applications (SPA).

4) Decisión tomada

Se adopta la **Opción C**. Se define la URL como la fuente de verdad única para la identificación de recursos. Todas las rutas de gestión se anidan bajo el patrón /eventos/:eventold. La obtención del identificador se centraliza mediante el hook useParams(). Se implementa una estrategia de transición manteniendo localStorage como fallback temporal exclusivamente para componentes en proceso de migración.

5) Consecuencias

- **Positivas:**
 - Mayor robustez ante recargas de página.
 - Soporte para navegación por historial (adelante/atrás) sin romper el contexto.
 - Posibilidad de implementar bookmarking y compartición de links de gestión.
- **Negativas / trade-offs:**
 - Requiere una refactorización de las rutas existentes en App.jsx.
 - Obliga a pasar el eventold a través de los enlaces de navegación.
- **Riesgos y mitigaciones:**
 - El riesgo de romper rutas antiguas se mitiga mediante una migración gradual y el uso del "fallback" temporal en la Sidebar.

6) Evidencia

- Implementado en la UT #3797 (Votaciones con jurado autenticado).
- Archivos modificados: App.jsx, InvitarJuradoPage.jsx, DashboardPage.jsx, EventSidebar.jsx.

ADR-006: Centralización del Estado del Evento mediante Contexto Global

Fecha: 21-04-2026

Sprint: S2

Estado: Aprobado

1) Contexto

La gestión de metadatos del evento (nombre, configuración visual, rol del usuario) se encontraba fragmentada. Esta dispersión generaba:

- **Desincronización de la UI:** Componentes transversales no reaccionaban en tiempo real a los cambios de contexto, afectando a la consistencia visual de la marca.
- **Prop Drilling y Redundancia:** Necesidad de leer y parsear localStorage repetidamente en múltiples niveles del árbol de componentes.
- **Dificultad de escalado:** La lógica para derivar estilos (colores según rol) estaba duplicada.

2) Opciones consideradas

- **Opción A (No hacer nada):** Seguir usando localStorage directamente en cada componente.
- **Opción B (Contexto Global):** Crear un EventContext de React para centralizar el estado y proporcionar métodos de actualización.
- **Opción C (Redux u otras):** Implementar una librería de gestión de estado externa.

3) Criterios de decisión

- **Consistencia:** El estado debe ser reactivo para toda la aplicación.
- **Simplicidad:** No añadir dependencias externas pesadas si el Core de React es suficiente.
- **Mantenibilidad:** Reducir la lógica duplicada en los componentes UI.

4) Decisión tomada

Se implementa la **Opción B (EventContext)**. Se crea un EventProvider que actúa como mediador de estado. Este contexto orquesta la información del evento y el rol del usuario, exponiendo un estado reactivo y métodos de actualización unificados. Se adopta un modelo de persistencia híbrida: el contexto gestiona la reactividad en memoria mientras sincroniza cambios en localStorage para garantizar la persistencia entre sesiones.

5) Consecuencias

- **Positivas:**
 - Consistencia visual garantizada: el cambio de tema o rol es atómico y global.
 - Reducción de la complejidad en componentes de vista, que ahora solo consumen el hook `useEvent()`.
 - Punto único para la lógica de estilos dinámicos.
- **Negativas / trade-offs:**
 - Ligero incremento en la profundidad del árbol de componentes (Provider nesting).
- **Riesgos:**
 - Posible sobrecarga de re-renders si el contexto crece excesivamente. Se mitigará mediante la fragmentación del estado si es necesario.

6) Evidencia

- Nuevo archivo: `Frontend/src/context/EventContext.jsx`
- Refactorización de: `App.jsx`, `EventSidebar.jsx`, `DashboardPage.jsx`, `OrganizerDashboard.jsx`.

ADR-007: Flujo de Onboarding de Jurados basado en Tokens de Invitación

Fecha: 21-04-2026

Sprint: S2

Estado: Aprobado

1) Contexto

La plataforma requiere un mecanismo de aprovisionamiento de cuentas con privilegios elevados (Jurados) que garantice la integridad y seguridad del sistema. El proceso de gestión de invitaciones presentaba los siguientes desafíos técnicos:

- **Seguridad y Autorización:** Necesidad de impedir la asignación no autorizada de roles de poder dentro de un evento, asegurando que solo usuarios validados por el Organizador puedan acceder.
- **Fricción en el Onboarding:** El sistema debe permitir la incorporación de usuarios externos que no poseen una cuenta previa en la plataforma de manera fluida.
- **Trazabilidad y Auditoría:** Necesidad de monitorizar el estado de las invitaciones (pendientes, aceptadas, expiradas) y permitir la gestión activa del ciclo de vida de cada invitación.

2) Opciones consideradas

- **Opción A (Asignación Directa por Email):** Registro inmediato del email en la lista de jurados. Se descarta por requerir que el usuario ya exista y por no garantizar el consentimiento explícito del invitado.
- **Opción B (Código de Acceso Compartido):** Uso de un PIN genérico por evento. Se descarta por ser poco seguro y propenso a filtraciones accidentales.
- **Opción C (Flujo basado en Tokens de un solo uso):** Generación de un enlace único vinculado a un secreto criptográfico y persistido temporalmente.

3) Criterios de decisión

- **Seguridad:** El acceso debe estar blindado por un token efímero de alta entropía.
- **UX:** Reducción de la carga cognitiva mediante un flujo de "un solo clic" que gestione la redirección y el registro automáticamente.
- **Determinismo:** Posibilidad de auditar y revocar el acceso en cualquier punto del flujo.

4) Decisión tomada

Se elige la **Opción C**. Se implementa una entidad invitaciones_pendientes que actúa como almacén de estado para el proceso de invitación (email, ID de evento y UUID/GUID).

El flujo se orquesta mediante la integración de **Resend** para el envío de notificaciones. Al consumir el token, la aplicación inyecta el contexto del evento directamente en el proceso de autenticación de Supabase, vinculando al usuario con el rol de Jurado de forma atómica tras el registro o inicio de sesión con éxito.

5) Consecuencias

- **Positivas:**
 - **Seguridad Robusta:** El token es consumible y de un solo uso, mitigando riesgos de interceptación.
 - **Onboarding Automatizado:** El usuario es redirigido y configurado sin intervención manual, mejorando la conversión de invitados.
 - **Gestión Centralizada:** Facilita la implementación de funciones de reenvío y revocación de privilegios desde el panel de administración.
- **Negativas / trade-offs:**
 - Acoplamiento con un proveedor de servicios de correo externo (Resend).
 - Necesidad de implementar procesos de limpieza para invitaciones expiradas en la base de datos.
- **Riesgos y mitigaciones:**
 - El riesgo de interceptación del token se mitiga mediante su invalidación inmediata tras el primer consumo exitoso.

6) Evidencia

- Persistencia: Tabla invitaciones_pendientes (Supabase).
- Lógica de Negocio: InvitacionPendienteRepository.cs, ResendEmailService.cs, JuradoService.cs.
- Interfaz: AcceptInvitePage.jsx y middleware en authApi.js.

ADR-008: Implementación de la extensión Fingerprinting para Unicidad de Voto y Seguridad

Fecha: 30-04-2026

Sprint: S2

Estado: Aprobado

1) Contexto

Votify permite la participación de usuarios no autenticados en categorías de voto público. Esta funcionalidad plantea un desafío crítico en términos de integridad de los resultados:

- **Prevención de Fraude:** Necesidad de garantizar la regla de "un voto por persona y categoría" sin comprometer la participación mediante registros obligatorios.
- **Insuficiencia de Métodos Tradicionales:** La identificación por dirección IP resulta ineficaz en redes compartidas y es fácilmente eludible mediante VPN. Por otro lado, el uso exclusivo de Cookies o LocalStorage es volátil, ya que el usuario puede evadir la restricción simplemente limpiando el historial o usando el modo incógnito.
- **Equilibrio entre Fricción y Seguridad:** El sistema debe identificar de forma persistente el dispositivo del votante sin recolectar información sensible ni requerir interacción adicional del usuario.

2) Opciones consideradas

- **Opción A (Restricción por IP):** Bloqueo basado en la dirección de red. Se descarta por su baja precisión en entornos institucionales o públicos y su facilidad de manipulación.
- **Opción B (Persistencia en Cliente):** Uso de Cookies de larga duración o LocalStorage. Se descarta como método primario debido a su fragilidad ante limpiezas de caché.
- **Opción C (Fingerprinting de Dispositivo):** Generación de un identificador basado en la entropía del navegador y hardware (resolución, renderizado de canvas, fuentes, zona horaria). Es significativamente más persistente y difícil de replicar.

3) Criterios de decisión

- **Determinismo y Unicidad:** Capacidad de generar el mismo identificador para el mismo dispositivo en sesiones distintas.
- **Minimización de Fricción:** El proceso de identificación debe ser transparente para el votante, eliminando la necesidad de login.
- **Resiliencia:** Capacidad para resistir intentos básicos de duplicación de identidad.
- **Privacidad por Diseño:** El método no debe almacenar datos personales, sino una representación hash irreversible de las características del sistema.

4) Decisión tomada

Se elige la **Opción C**. Se implementa la tecnología de **Fingerprinting** mediante la integración de la librería FingerprintJS en el frontend.

El sistema genera un identificador único (hash) que se adjunta a cada petición de voto. El backend orquesta la validación a través de la entidad `registro_votos_publicos`, que actúa como registro de auditoría vinculando el evento, la categoría y el hash del dispositivo. Se adopta una estrategia de **seguridad en capas**, utilizando `LocalStorage` como *fallback* en caso de que el script de fingerprinting sea bloqueado por políticas estrictas del navegador.

5) Consecuencias

- **Positivas:**
 - **Integridad del Voto Popular:** Se garantiza una alta correlación entre dispositivo y voto único, aumentando la legitimidad de los resultados.
 - **Experiencia de Usuario Fluida:** Permite el voto masivo y anónimo sin barreras de registro.
 - **Persistencia Multired:** El identificador permanece invariable, aunque el usuario cambie de red Wi-Fi o reinicie su router.
- **Negativas / trade-offs:**
 - Dependencia de la capacidad del navegador para ejecutar scripts de análisis de entorno.
 - Posibilidad teórica marginal de colisiones de hash en dispositivos con configuraciones idénticas.
- **Riesgos y mitigaciones:**
 - El bloqueo por extensiones *anti-tracking* se mitiga mediante la persistencia redundante en el almacenamiento local del navegador.

6) Evidencia

- **Lógica Frontend:** Implementación del hook personalizado `useFingerprint.ts`.
- **Validación Backend:** Lógica de control de duplicados en `VotoService.cs`.
- **Persistencia:** Definición del esquema `registro_votos_publicos` en la base de datos.
- **Calidad:** Cobertura de pruebas en `useFingerprint.test.ts` y pruebas de integración de servicios de voto.

Anexo: Metodología de Generación de los ADR

Para asegurar que la documentación técnica reflejara fielmente la evolución del proyecto, se adoptó un flujo de trabajo con la implementación de las UTs. El proceso seguido por el equipo fue el siguiente:

1. **Implementación Técnica:** El desarrollo del código y la resolución de problemas.
2. **Identificación de Decisiones:** Una vez finalizada y testeada una funcionalidad que implicara un cambio arquitectónico o de diseño relevante, el equipo identificaba la necesidad de registrar dicha decisión.
3. **Generación Asistida del ADR:** Se utilizó la capacidad de síntesis de la IA para transformar el contexto de la conversación y el código recién generado en un documento estructurado siguiendo el estándar ADR. Este proceso garantiza que la documentación sea precisa y coherente con el código final en el repositorio.

Nota: El Prompt 3 detallado en el AI Usage Log de esta memoria sirve como ejemplo directo de este proceso, mostrando cómo se generó específicamente el ADR-008.

5. Uso y Documentación de IA Generativa - AI Usage Log

5.1. Herramientas usadas

- **Herramienta/modelo:** ChatGPT-4o, GitHub Copilot, Gemini, Antigravity, Gemini CLI
- **Para qué se uso:**
 - Planificación y refinamiento de requisitos.
 - Arquitectura y diseño del sistema (Backend y Frontend).
 - Generación de lógica de negocio y código.

5.2. Prompts clave

- **Prompt 1:** en mi proyecto de votify, estoy haciendo una funcionalidad, en la que, introduzco las opciones de dar obligatoriedad por parte del organizador a los comentarios globales, tanto de los criterios que tienen los proyectos, como de los proyectos en general, necesito que me ayudes para ver como podria introducir los patrones de diseño software estrategia y decorador, explicame la manera en las que los puedo meter, y en que parte deberia implementarlos
- **Prompt 2:** *Imagen*. Quiero que me recrees esta página en ControlVotacionesPage (Respeto el sidebar, encabezado y color del organizador). Condiciones: usa los componentes EstadoCategoriaCard y OpcionesBar. En las tarjetas quiero no se mostrará el countdown si la categoria tienen las dos fechas a null, en ese caso solo se pondra el texto "Sin horas prefijadas". No se podrán seleccionar las categorias que esten finalizadas. No quiero los titulos de distintos colores. En cuanto al bar de opciones , lo quiero en naranja. Si aprete el boton de activar y aun no era la fecha ni se guardara el nuevo tiempo de inicio y se verá reflejado en el countdown. Si le da al boton finalizar, le saldra un recuadro modal donde diga "Cuidado, si finalizas no podrás reabrir la votación" y se guardara o actualizará en bd la nueva fechaFin. Si le da al boton pausa en CategoriaCard no se podrá apretar el boton de votar y le dira que esta pausada y en votoService no se guardara cualquier voto que entre en estado "Pausada" y se desactivará el boton de confirmar hasta que se vuelva a reanudar. EL boton reanudar solo se podrá apretar si esta en pausa y su funcion es devolver a estado activa a la categoria. Si se pausa , el countdown seguirá ya que este solo refleja cuanto queda hasta el cierre programado. Ten en cuenta que ahora el estado default es "Pendiente", ya que antes se ponian a Activa simple.

- **Prompt 3:** Actúa como un Arquitecto de Software. Analiza la implementación de seguridad para el voto público (fingerprinting, almacenamiento en BD y validaciones en el servicio) y redacta el `ADR-008` siguiendo la plantilla. La decisión debe centrarse en el uso de `FingerprintJS` para garantizar la unicidad del voto sin login. Justifica por qué se eligió sobre cookies o IP, detalla las consecuencias y asegura que la evidencia coincida con los archivos actuales (@useFingerprint.ts, @VotoService.cs).

5.3. Salidas relevantes

- **Prompt 1:** Me explicó un poco como funcionan los dos patrones, así como crearlos y situarlos en el proyecto, además de la funcionalidad que tendrían cada uno en la funcionalidad dada y el flujo que se seguiría con ellos
- **Prompt 2:** Me creo un plan en el que básicamente me recrea una pantalla en la que puedo alterar los estados de las categorías. Me creó y modificó los archivos necesarios en frontend (hooks, useConfigTiempo, y configuracionesApi) y en backend me modificó el votoService , creo Dtos y métodos en los repositorios.
- **Prompt 3:** Generación de un documento estructurado en Markdown que explica el razonamiento técnico tras el sistema de seguridad de votos, incluyendo las posibles alternativas.

5.4. Qué aceptamos y qué rechazamos

- El **prompt 1** fue aceptado, debido a su aportación positiva a la hora de implementar nuevos patrones de diseño al proyecto
- El **prompt 2** fue corregida porque no entendió bien como representar bien el countdown y tenía problemas con los tiempos si se alteraban las horas de manera manual. Además, había problemas de refresco de los estados en las tarjetas de las categorías. Por tanto, se le notificó para su arreglo.
- En el **prompt 3** se aceptó el razonamiento técnico y la comparativa de métodos de seguridad, pero se rechazaron las referencias a archivos genéricos y se realizó una corrección manual de la sección de evidencia para vincularla estrictamente a la tabla registro_votos_publicos y a la implementación real del proyecto.

5.5. Cómo lo verificamos

- [X] **Tests unitarios:**
 - Para el prompt 1 se utiliza ComentarioValidationTests.cs usando xUnit para comprobar el funcionamiento del patron strategy.
- [X] **Experimento / medición:**
 - Para el prompt 2 se fue comprobando manualmente cada uno de los estados, para ver posibles fallos visuales, porque hay cosas que funcionaban, pero hacían ruido al verse y se fue mejorando para tener lo buscado.
- [X] **Consulta a documentación oficial:**
 - Revisión de la documentación de Supabase C# Client y React hooks.
 - Para el prompt 3, se hizo contraste con la plantilla de documentación de arquitectura (ADR)
- [X] **Revisión por pares:**
 - Para el prompt 1 el código de los 2 patrones fue revisado por parte del backup, verificando que se hace uso de buenas prácticas en su implementación y de su correcto funcionamiento testeando el flujo de trabajo
 - Para el prompt 2 el backup comprobó que hay coherencia en la lógica de los cambios de los estados, y que no había fallos visuales o anomalías.
 - Para el prompt 3 se hizo una validación por parte del backup para asegurar que la decisión reflejada en el log coincide con la lógica implementada en el código.

5.6. Resultado final / decisión humana

- **Prompt 1:** La ayuda de este prompt hizo que se hiciera un push de la solución con ambos patrones para la funcionalidad y, una posterior pull request que quedó plasmada en la rama main del proyecto Votify.

Código del commit: 9b03dc43c6a358734b0625a58eadfff50d3eb6e

- **Prompt 2:** Implementado tras corregir la falta de actualización de los estados en las tarjetas EstadoCategoriaCard y el problema con las horas (countdown y guardado de fechas en bd)

Código del commit: 44b6e4c86eaa1ae4282581a8458b5502b068068f

- **Prompt 3:** El grupo formalizó el uso de Fingerprinting como el estándar de seguridad para la participación anónima. La decisión final fue priorizar la integridad del voto popular sin añadir fricción al usuario, asumiendo el trade-off de la dependencia de una librería externa. Se realizó además una limpieza de la documentación técnica obsoleta para mantener el histórico de diseño enfocado exclusivamente en la arquitectura vigente.

Código del commit: 4110228dc70dcf53f00320f6194c030d05e52dd1